

Code Review Agent:

Otonom Araç Orkestrasyonu ve İteratif Akıl Yürütme Temelli Gelişmiş Kod İnceleme Sistemi

Teknik Tasarım ve Akademik Analiz Raporu

Hazırlayanlar: Abdullah Sait AVCI - G231210035

Fatih Kaya - G231210072

Bölüm: Bilgisayar Mühendisliği

Üniversite: Sakarya Üniversitesi

Tarih: 12 Mayıs 2026

Versiyon: 1.3.0

Konu: LLM Tabanlı Otonom Ajan Mimarisi, Statik Analiz
Entegrasyonu ve Uzun Süreli Bellek Yönetimi

Uygulama:  Code Review Agent

GitHub:  GitHub - AbdullahSaitAvci/code-review-agent

1. Özet

Bu teknik rapor, yazılım mühendisliği süreçlerinde kod kalitesini artırmak ve mühendislik standartlarını otomatize etmek amacıyla geliştirilen "Code Review Agent" sisteminin mimari tasarımını ve uygulama detaylarını ele almaktadır. Sistem, geleneksel statik analiz araçlarının (Pylint, Flake8) deterministik doğasını, Büyük Dil Modellerinin (LLM) semantik analiz yetenekleriyle birleştiren hibrit bir yapı üzerine kurulmuştur. Projenin merkezinde, Anthropic'in claude-sonnet-4-6 ve claude-sonnet-4-5 modelleri tarafından yönetilen otonom bir araç kullanım döngüsü (agentic loop) yer almaktadır. Bu döngü sayesinde sistem, kodun sadece sözdizimini değil, aynı zamanda yapısal karmaşıklığını ve tasarım desenlerini de inceleyebilmektedir. Rapor; sistem mimarisi, araç orkestrasyonu, kalıcı bellek yönetimi ve yapay zeka destekli geliştirme süreçlerindeki mühendislik kararlarını niteliksel analizler ve gerçek sistem davranışları ışığında detaylandırmaktadır.

2. Abstract

This research report presents a comprehensive technical analysis of the "Code Review Agent," an autonomous AI system designed to conduct sophisticated code inspections through iterative reasoning and deterministic tool orchestration. The architecture leverages state-of-the-art Large Language Models (LLMs), primarily claude-sonnet-4-6 and claude-sonnet-4-5, as central reasoning engines that interact with a specialized toolbelt comprising Pylint, Flake8, and custom AST-based complexity parsers.

The system implements a robust Reasoning-Act-Observe (ReAct) style iterative loop, facilitated by a native provider abstraction layer that bridges Anthropic's tool-use protocol with OpenAI-compatible function calling. A significant architectural achievement of the project is the integration of longitudinal persistent memory, which utilizes a JSON-based user profiling mechanism to track recurring coding anti-patterns. This report details the engineering rigor applied to tool call synchronization, subprocess isolation for security, and the optimization of context window management through ephemeral prompt caching. The study concludes with a deep dive into AI-assisted software engineering (AI-ASE) methodologies used during the development lifecycle, highlighting the transition from raw LLM outputs to validated, production-grade engineering reviews.

3. Giriş

Yazılım mühendisliği disiplininde kod incelemesi (code review), teknik borcun yönetilmesi ve yazılım güvenilirliğinin sağlanması için kritik bir süreçtir. Ancak manuel incelemeler; insan faktöründen kaynaklanan tutarsızlıklara, yüksek zaman maliyetine ve uzmanlık darboğazlarına açıktır. Mevcut statik analiz araçları ise sadece kural tabanlı hataları yakalayabilmekte, kodun "neden" hatalı olduğuna dair pedagojik bir rehberlik sunamamaktadır.

"Code Review Agent" projesi, bu kısıtları aşmak amacıyla **Autonomous Agentic Workflow** paradigmasını temel alarak geliştirilmiştir. Bu sistemde yapay zeka modeli, kullanıcıya sadece statik bir metin dönen pasif bir birim değil; aksine, inceleme sürecinde hangi analizin gerekli olduğuna karar veren, bu analizleri "tools.py" içerisindeki Python fonksiyonları aracılığıyla gerçek dünya üzerinde yürüten ve elde ettiği yapılandırılmış verileri sentezleyen bir orkestratördür. Sakarya Üniversitesi Bilgisayar Mühendisliği bünyesinde hazırlanan bu çalışma, yapay zeka ve klasik yazılım analizi araçlarının sinerjisini akademik ve teknik bir perspektifle ortaya koymaktadır.

Sistemin esnek yapısı, Anthropic'in sonnet-4-6 gibi gelişmiş modellerinden, OpenRouter üzerinden erişilen açık kaynaklı alternatiflere kadar geniş bir model yelpazesini destekler. Bu destek, "openrouter_core.py" içindeki **Bridge Pattern** yaklaşımıyla sağlanarak, farklı sağlayıcıların araç kullanım formatları arasındaki uyumsuzluklar mühendislik seviyesinde çözülmüştür.

4. Problem Tanımı

Kod inceleme otomasyonunda karşılaşılan ve bu projenin teknik mimarisini şekillendiren temel mühendislik problemleri şunlardır:

4.1. Semantik ve Statik Analiz Senkronizasyonu

Büyük Dil Modelleri (LLM), kodun genel amacını anlamada başarılı olsa da, yapısal metriklerde (nesting depth, line count) halüsinasyon riski taşımaktadır. Mühendislik raporlarının ise deterministik verilere dayanması şarttır. “tools.py” içerisinde tanımlanan “analyze_complexity” ve “run_static_analysis” fonksiyonları, bu riskleri minimize etmek için modelin olasılıksal (probabilistic) çıktılarını denetleyen ve analitik süreci deterministik verilerle besleyen birer tekil gerçeklik kaynağı (single source of truth) olarak tasarlanmıştır."

4.2. Çoklu Sağlayıcı ve Araç Şeması Uyumsuzlukları

Proje, Anthropic SDK ve OpenRouter üzerinden farklı modelleri desteklemektedir. Ancak, Anthropic'in beklediği araç şeması yapısı ile OpenAI uyumlu “function-calling” protokolü arasında yapısal farklar mevcuttur. Bu durum, sistemin taşınabilirliğini ve model değişiminde analiz kalitesinin korunmasını zorlaştıran bir orkestrasyon problemidir. Sistem, bu sorunu “_to_openai_tool” dönüşüm mantığıyla aşarak yüksek bir soyutlama başarısı elde etmiştir.

4.3. Süreklilik ve Kalıcı Bellek Eksikliği

Mevcut AI çözümlerinin çoğu her isteği bağımsız bir işlem (stateless) olarak görür. Oysa bir yazılımcının gelişimi için geçmişteki hatalarının (örneğin sürekli unutulan docstring'ler veya yüksek karmaşıklığa sahip if blokları) takip edilmesi gerekir. “user_profile.json” üzerinden sağlanan uzun dönemli bellek (long-term memory), bu "öğrenen sistem" ihtiyacını karşılamayı amaçlar.

4.4. Kaynak İzolasyonu ve Güvenlik (Subprocess Enforcing)

Statik analiz araçlarının (pylint, flake8) yerel makinede çalıştırılması, süreç yönetimi ve güvenlik riskleri barındırır. Kodun analiz sırasında sonsuz döngüye girmesi veya aşırı kaynak tüketmesi riskine karşı, “tools.py” içerisinde “_TIMEOUT = 15” gibi katı sınırlandırmalar ve geçici dosya yönetimi (tempfile.NamedTemporaryFile) mekanizmaları uygulanmıştır.

5. Literatür Araştırması ve Teknik Arka Plan

Kod inceleme ajanlarının tasarımı, yapay zeka ve yazılım mühendisliği literatüründeki üç ana akımın kesişim noktasında yer almaktadır:

1. **Static Program Analysis (SPA):** AST (Abstract Syntax Tree) parsing ve veri akış analizi üzerine kurulu klasik linter teknolojileri. Bu araçlar yüksek kesinlik (precision) ancak düşük bağlamsal derinlik sunar.
2. **Large Language Models for Code (Code-LLMs):** claude-sonnet-4-6 gibi modellerin kod anlama kapasiteleri. Bu modeller yüksek semantik anlayış sunarken potansiyel determinizm sorunları taşır.
3. **Agentic Orchestration:** Bir LLM'nin dış araçları (external tools) kullanabilme yeteneğidir. Literatürde **ReAct (Reasoning and Acting)** olarak tanımlanan bu yaklaşım, modelin yeteneklerini dosya okuma, kod çalıştırma ve statik analiz yapma gibi fiziksel eylemlerle genişletir.

Code Review Agent, bu üç alanı "tools.py" içerisindeki araç setleri ve "core.py" içerisindeki iteratif döngü aracılığıyla sentezleyerek literatürdeki "Hibrit Kod Zekası" (Hybrid Code Intelligence) yaklaşımını standardize etmektedir. Özellikle Anthropic'in ephemeral caching (geçici önbelleğe alma) özelliği, sistemin her iterasyonda bağlam penceresini (context window) verimli kullanmasını sağlayarak gecikme sürelerini optimize etmektedir.

6. Agent Mimarisi ve Tool Use Akışı

Code Review Agent, klasik bir chatbot'tan farklı olarak **Reasoning-Act-Observ** (**ReAct**) döngüsüyle çalışır. Bu yapı sayesinde model, bir sonuca varmadan önce mevcut bilgiyi analiz eder (Reasoning), eksik veriyi tamamlamak için gerekli araçları çağırır (Act) ve elde ettiği bulguları değerlendirerek (Observation) süreci yineler.

6.1. Agentic Loop Mekanizması

Döngü, core.py içerisinde `_MAX_ITERATIONS = 20` kısıtıyla korunur. Bu, ajanın sonsuz döngüye girmesini (agentic loop failure) engelleyen kritik bir mühendislik bariyeridir.

Pseudo-Code Analizi:

```
while iterations < _MAX_ITERATIONS:
    # Model'e mevcut mesaj geçmişini ve araç setini gönder
    response = client.messages.create(model, messages, tools, system_prompt)

    # 1. Aşama: Model yanıtı analiz et (Reasoning)
    messages.append({"role": "assistant", "content": response.content})

    if response.stop_reason == "end_turn":
        # Model incelemeyi tamamladı, raporu döndür
        return final_markdown_review

    elif response.stop_reason == "tool_use":
        # 2. Aşama: Araç çağrılarını yürüt (Acting)
        tool_results = []
        for block in response.content:
            if block.type == "tool_use":
                result = _dispatch_tool(block.name, block.input)
                tool_results.append(result)

        # 3. Aşama: Sonuçları modele geri besle (Observation)
        messages.append({"role": "user", "content": tool_results})
```

Bu akışın klasik chatbot'lardan farkı, **Recursive Orchestration** yeteneğidir. Model, pylint sonucunda bir karmaşıklık hatası gördüğünde, bir sonraki iterasyonda `analyze_complexity` aracını çağırmaya karar verebilir. Bu, statik bir script değil, dinamik bir karar ağacıdır.

7. Bellek Sistemi (Memory Architecture)

Sistem, kısa süreli (session-based) ve uzun süreli (persistent) olmak üzere iki tip bellek kullanır.

7.1. Kalıcı Bellek: user_profile.json

"data/user_profile.json" dosyası, kullanıcının gelişimini takip eden yapılandırılmış bir veritabanı görevi görür.

Veri Yapısı:

- **recurring_issues:** Pylint/Flake8 hata kodlarını (ör. "C0116", "E501") anahtar olarak tutan ve her hatada sayacı artıran bir sözlüktür.
- **review_count:** Toplam inceleme sayısını tutarak istatistiksel bağlam sağlar.

7.2. Mühendislik Kararı: Neden JSON?

Küçük ve orta ölçekli ajan sistemlerinde Relational Database (RDBMS) yerine JSON tercih edilmesinin temelinde şu mühendislik gerekçeleri yatmaktadır:

1. **Düşük Gecikme (Low Latency):** Yerel dosya tabanlı erişim, ağ üzerinden veritabanına bağlanma zorunluluğunu ortadan kaldırarak gecikmeyi minimize eder.
2. **Şema Esnekliği (Schema Flexibility):** Ajanın dinamik olarak yeni veri türleri veya parametreler üretmesi durumunda, JSON yapısı önceden tanımlanmış bir şemaya ihtiyaç duymadan bu değişime anında uyum sağlar.
3. **Eşzamanlılık Yönetimi (Concurrency Management):** Streamlit mimarisi gereği her kullanıcı oturumu izoledir; JSON tabanlı kalıcılık (persistence) bu ölçekteki işlemler için yeterli veri yalıtımını ve bütünlüğünü sunar.
4. **Ödünleşim (Tradeoff):** Ölçeklenebilirlik (scaling) açısından JSON, binlerce eş zamanlı kullanıcıda kilitleme (file locking) problemleri yaratabilir. Ancak bireysel bir mühendislik asistanı için bu "simplicity vs complexity" dengesinde rasyonel bir seçimdir.

8. Çoklu Sağlayıcı Mimarisi ve API Entegrasyonları (Multi-Provider Abstraction)

Code Review Agent, tek bir model sağlayıcısına bağımlı kalmanın yaratacağı riskleri (vendor lock-in) minimize etmek ve farklı maliyet/performans dengeleri sunabilmek amacıyla modüler bir **Sağlayıcı Soyutlama Katmanı (Provider Abstraction Layer)** üzerine inşa edilmiştir. Sistem, “agent/core.py Anthropic” ve “agent/openrouter_core.py OpenRouter/OpenAI” modülleri üzerinden iki farklı protokolü desteklemektedir.

8.1. Bridge Pattern: Anthropic ↔ OpenAI Tool Dönüşümü

Sistemin en özgün mühendislik çözümlerinden biri, “openrouter_core.py” içerisinde yer alan “_to_openai_tool” fonksiyonudur. Anthropic SDK, araç şemalarını (tool schemas) “input_schema” anahtarı altında beklerken, OpenAI uyumlu API'lar (OpenRouter dahil) parameters anahtarını kullanmaktadır.

Teknik Dönüşüm Mantığı:

```
def _to_openai_tool(schema: dict) -> dict:
    return {
        "type": "function",
        "function": {
            "name": schema["name"],
            "description": schema["description"],
            "parameters": schema["input_schema"], # Şema uyumlulaştırma noktası
        },
    }
```

Bu yaklaşım, “agent/tools.py” içerisinde tanımlanan şemaların tek bir formatta tutulmasını sağlayarak "Single Source of Truth" (Tekil Gerçeklik Kaynağı) prensibini korur. Bu sayede yeni bir araç eklendiğinde, her iki sağlayıcı için ayrı şemalar yazma zorunluluğu ortadan kalkar.

Bölüm 8.2. Model Karşılaştırma ve Mühendislik Değerlendirmesi

Model	Sağlayıcı	Niteliksel Analiz	Gecikme (Latency)	Araç Kullanım Kararlılığı
claude-sonnet-4.6	Anthropic	Uzun bağlam senaryolarında daha tutarlı reasoning üretmiş; pedagojik çıktı kalitesi en yüksek seviyede bulunmuştur.	Düşük (Caching destekli)	Yüksek (Daha stabil tool-use davranışı)
claude-sonnet-4.5	Anthropic	Karmaşık mimari analizlerinde yüksek bağlamsal derinlik ve çözüm önerisi tutarlılığı sergilemiştir.	Orta	Yüksek
claude-haiku-4.5	Anthropic	Hızlı stil kontrollerinde verimli; ancak çok adımlı araç zincirlerinde akıl yürütme sapmaları gözlemlenebilir.	Çok Düşük	Orta-Yüksek
gpt-oss 120B	OpenRouter	JavaScript, Java ve Python analizlerinde yapılandırılmış tablo formatında çıktı üretmiş; araç çağırma hiyerarşisine genel olarak uyum sağlamıştır.	Orta-Yüksek (Kuyruk kaynaklı)	Orta-Yüksek
Llama 3.3 70B	OpenRouter	Açık kaynaklı modeller arasında en dengeli araç çağırma hiyerarşisine sahip model olarak belirlenmiştir.	Değişken (Kuyruk kaynaklı)	Orta
Nemotron / Gemma	OpenRouter	Düşük maliyetli/ücretsiz modellerde tool-call davranışı, özellikle karmaşık JSON şemalarında değişkenlik göstermiştir.	Değişken	Orta-Düşük

9. Prompt Engineering ve Talimat Hiyerarşisi

Sistemin başarısı, "agent/prompts.py" dosyasında tanımlanan **Instruction Hierarchy** (Talimat Hiyerarşisi) yapısına dayanmaktadır. "SYSTEM_PROMPT" metni, ajanın davranışını belirleyen katı kurallar silsilesi içerir.

9.1. Deterministik Akış Zorlaması (Deterministic Prompting)

Modelin araçları rastgele değil, belirli bir mühendislik mantığıyla çağırması için "İnceleme Akışı" (Review Flow) tanımlanmıştır. Bu akışta;

1. İlk olarak "get_user_profile" çağrılarak bağlam kurulması,
2. Ardından dil bazlı statik analizin yapılması,
3. Son olarak "update_user_profile" ile kalıcı belleğin güncellenmesi zorunlu kılınmıştır.

9.2. Hallüsinasyon Azaltma Stratejileri

Ajanın kodda olmayan hataları uydurmasını (hallucination) engellemek için, "Sadece araçlardan gelen bulgularla konuş" talimatı verilmiştir. Modelin kendi dil bilgisini kullanması, yalnızca araçların desteklenmediği (Python dışı diller) senaryolarda bir "fallback" (yedek plan) olarak tanımlanmıştır. Ayrıca, pedagojik yaklaşım gereği "önce/sonra" kod örnekleri sunması istenerek, önerilerin somut verilere dayanması sağlanmıştır.

10. Statik Analiz ve Karmaşıklık Analiz Katmanları

Projenin mühendislik değerini oluşturan temel katman; LLM'in **olasılıksal (probabilistic) çıktıları**nı deterministik metriklerle **denetleyen** ve süreci gerçek verilerle **temellendiren (grounding)** statik analiz katmanıdır.

10.1. Subprocess Yönetimi ve Güvenlik

"agent/tools.py" modülü, pylint ve flake8 araçlarını "subprocess.run" komutu ile izole edilmiş bir süreçte çalıştırır.

1. **Timeout Enforcement:** _TIMEOUT = 15 kısıtı, analiz edilen kodun (veya aracın) işletim sistemini kilitlenmeye sokmasını engeller.
2. **Temporary Lifecycle:** Kullanıcı kodu asla kalıcı olarak diskte tutulmaz; "tempfile.NamedTemporaryFile" ile bellekte oluşturulup analiz biter bitmez imha edilir.

10.2. AST Traversal ve Karmaşıklık Algoritması

Kodun yapısal derinliği, LLM'in tahminlerine bırakılmak yerine Python'ın ast (Abstract Syntax Tree) modülü ile ağaç yapısında taranır.

Nesting Depth Hesaplama Mantığı: Ajan, "_nesting_depth" fonksiyonu ile kodun içindeki If, For, While ve With bloklarını rekürsif (özyinelemeli) olarak ziyaret eder. Gerçek sistem çıktılarında, karmaşık kodlarda tespit edilen 4-6 seviyeli derinlikler,

modelin "bu fonksiyon refaktör edilmeli" kararını rasyonel bir temele oturtur.

11. Çok Dilli Destek ve Dosya Yönetimi

Sistem, Python odaklı geliştirilse de genişletilebilir bir dil destek yapısına sahiptir. app.py üzerinde yer alan `_LANG_MAP` sözlüğü, dosya uzantılarını (.js, .cpp, .java, vb.) otomatik olarak algılayarak ajana gerekli dil bağlamını iletir.

1. **Haritalama Mekanizması (Mapping):** Uzantı tabanlı bu tespit süreci, ajanın çalışma anında (runtime) hangi statik analiz araçlarını aktive edeceğini belirleyen bir karar mekanizması işlevi görür.
2. **Hata ve İstisna Yönetimi (Fallback Logic):** Python dışındaki dillerde "run_static_analysis" aracının boş değer döndüreceği bilgisi prompts.py aracılığıyla modele ön koşul olarak tanımlanmıştır. Bu sayede modelin, dile özgül fonksiyonların o dil dışı dillerde kullanımı sonucu oluşacak geçersiz araç çağırısı (invalid tool call) yapması mimari seviyede engellenmiştir."

12. Streamlit UI Mimari ve UX Kararları

Kullanıcı arayüzü, ajanın akıl yürütme (reasoning) süreçlerini ve statik analiz çıktılarını eş zamanlı olarak sunan bir 'Yapay Zeka İzleme Paneli' (AI Dashboard) mimarisiyle yapılandırılmıştır.

12.1. Session State Lifecycle

Streamlit'in "stateless" yapısı, "st.session_state" kullanılarak "stateful" bir oturuma dönüştürülmüştür. messages geçmişi her etkileşimde korunarak, ajanın çok adımlı (multi-hop) akıl yürütme sürecinin UI tarafında kesintiye uğraması engellenmiştir.

Bölüm 12.2. Quality Score Görselleştirme

Teknik Analiz ve İmplementasyon

Sistem, inceleme kalitesini kullanıcıya somut bir metrikle sunmak amacıyla uygulama katmanında dinamik bir puanlama algoritması işletir. Gerçek gerçekleştirimde (implementation) skor, ajanın gerçekleştirdiği araç kullanım yoğunluğuna (tools_used) göre doğrudan app.py içerisinde belirlenmektedir.

Özellikle update_user_profile aracının çağırılma sıklığı, koddaki "tekrarlayan hata" yoğunluğunu ve dolayısıyla teknik borcu temsil etmektedir. Puanlama mekanizması şu formül uyarınca sayısal bir değere dönüştürülür:

$$Score = \max(0, \text{round}(10 - n \cdot 1.5))$$

(Burada n , ilgili analiz oturumunda gerçekleştirilen profil güncelleme sayısını

temsil eder.)

Bu mimari kurguda “_quality_score_html” fonksiyonu, herhangi bir iş mantığı (business logic) barındırmaz. Fonksiyonun yegâne sorumluluğu, uygulama katmanı tarafından üretilen verinin HTML/CSS tabanlı bir arayüz bileşeni (widget) olarak render edilmesidir. Bu yaklaşım, **İş Mantığı** ile **Sunum Katmanının (Presentation Layer)** birbirinden izole edilmesi prensibine dayanarak sistemin test edilebilirliğini ve sürdürülebilirliğini artırmaktadır.

13. Yapay Zeka Destekli Geliştirme Süreci (AI-ASE)

Projenin yaşam döngüsü, Yapay Zeka Destekli Yazılım Mühendisliği (AI-ASE) prensiplerinin uçtan uca uygulandığı bir metodolojiyle yönetilmiştir. Geliştirme süreci; LLM çıktılarının doğrudan kullanımı yerine; doğrulama, iteratif iyileştirme ve mühendislik denetimi aşamalarını içeren bir iş akışına dayanmaktadır.

13.1. Model Roller ve Spesifik Katkı Analizi

Geliştirme aşamasında farklı modellerin mimari yeteneklerinden, projenin farklı katmanlarında faydalanılmıştır:

- Claude Sonnet Serisi (Mimari ve Core Mantığı):** “agent/core.py” içerisinde yer alan otonom döngü ve stop_reason yönetimi gibi yüksek seviyeli akıl yürütme gerektiren kısımlar claude-sonnet-4-5 ve 4-6 modelleriyle tasarlanmıştır. Özellikle Anthropic’in araç kullanım (tool-use) protokolündeki hata toleransı mekanizmaları bu modellerin mimari üretimleri sonucunda şekillenmiştir.
- Gemini (Sistem Analizi ve Dokümantasyon):** Projenin bütünsel teknik bağlamının çıkarılması ve kod dosyaları arasındaki ilişkilerin akademik bir dille belgelenmesinde Gemini’nin geniş bağlam penceresi yeteneklerinden yararlanılmıştır.
- ChatGPT (Meta-Prompt Üretimi):** Gemini ve Claude çıktılarını sentezleyerek gereksinim analizi ve meta-prompt üretimi süreçlerinde stratejik bir köprü görevi üstlenmiştir.

Bölüm 13.2. Mühendislik Denetimi ve İnsan-Döngüde

(Human-in-the-Loop) Yaklaşımı

Kod Validasyonu: Üretilen araç entegrasyonları, tests/test_tools.py bünyesindeki birim testleri ile doğrulanmıştır. Test stratejisi; niceliksel bir kod kapsamından ziyade, kritik araç davranışlarının doğrulanmasına ve AST ayrıştırma mantığının geçerliliğine odaklanmıştır. Bu sayede, AI tabanlı üretimin doğasında bulunan yapısal riskler kontrol altına alınarak sistemin operasyonel güvenilirliği artırılmıştır.

14. Performans ve Latency Analizi

Ajan tabanlı mimarilerin en kritik performans darboğazı, modelin bir sonuca varmak için yürüttüğü çok adımlı (**multi-hop**) akıl yürütme döngüsüdür. Bu sistemde gecikme ve maliyet optimizasyonu sağlamak amacıyla "erken durdurma" (early stopping) ve "bağlam yönetimi" gibi mühendislik stratejileri uygulanmıştır.

Bölüm 14.1. Gecikme Kaynakları ve Optimizasyonlar

İteratif Döngü Maliyeti (Iterative Loop Cost): Her bir araç çağrısı bağımsız bir API gidiş-dönüşü (**round-trip**) gerektirmektedir. Saha gözlemlerine göre; ajanın `get_user_profile` ile bağlam kurduğu, `run_static_analysis` ve `analyze_complexity` ile derin analiz yürüttüğü ve son aşamada `update_user_profile` ile kalıcı belleği güncellediği tam kapsamlı senaryolarda döngü genellikle **7-10 iterasyon** bandında tamamlanmaktadır.

Analitik Doğruluk vs. Hız: Klasik chatbot etkileşimlerine kıyasla toplam yanıt süresinin (latency) artması, deterministik verilere dayalı **didaktik** (öğretici) bir rapor üretimi için kaçınılmaz bir akıl yürütme maliyeti olarak kabul edilmektedir.

14.2. Token Growth ve Context Window Yönetimi

Döngü ilerledikçe messages listesi büyümekte ve bağlam penceresi (context window) baskısı artmaktadır. `_MAX_ITERATIONS = 20` limiti, hem maliyetin kontrolsüz artışını engellemekte hem de modelin odağını (focus) kaybetmesini önlemektedir.

15. Güvenlik Önlemleri ve Hata Yönetimi

istem, dış kaynaklı (kullanıcı tarafından sağlanan) kod bloklarını analiz ettiği ve yerel işletim sistemi araçlarını tetiklediği için **katmanlı bir güvenlik izolasyonuna** ihtiyaç duyar.

15.1. Süreç İzolasyonu ve Subprocess Güvenliği

"agent/tools.py" dosyasındaki tüm yerel analizler subprocess modülü ile gerçekleştirilir.

- Kısıtlı Yetki:** Analiz araçları (pylint, ast) kodun kendisini çalıştırmaz (exec/eval yapmaz), sadece statik olarak metni tarar. Bu, "Remote Code Execution" (RCE) riskini minimize eder.
- Timeout Enforcing:** Analiz edilen dosyanın kasıtlı veya hatalı bir şekilde aşırı kaynak tüketmesine (örneğin devasa dosyaların taranması) karşı timeout mekanizması bir savunma katmanıdır.

15.2. Exception Boundaries ve Veri Gizliliği

Hata Yalıtımı (Exception Boundaries): Araç yürütme aşamasında meydana gelebilecek düşük seviyeli hatalar (JSONDecodeError, OSError vb.), ajanın yorumlayabileceği **yapılandırılmış JSON formatına** dönüştürülerek yakalanır. Bu durum, beklenmedik hataların ana sistem akışını bozmasını (system crash) önleyen bir hata izolasyonu sağlar.

Kalıcı Olmayan Veri Yönetimi (Data Ephemerality): Analiz sürecinde oluşturulan geçici dosyalar tempfile modülü ile yönetilir. İşlem tamamlandığında bu veriler unlink metoduyla güvenli bir şekilde silinir. Bu sayede, kullanıcı kodlarının disk üzerinde kalıcı bir iz bırakması engellenerek **veri gizliliği** korunur.

16. Prompt Mühendisliği ve Mimari Uygulama

Bölüm 9'da detaylandırılan stratejik yönergeler, uygulama katmanında somut operasyonel talimat setlerine dönüştürülmüştür. Bu bölüm, sistemin otonom karar mekanizmasını yöneten **sistem prompt mimarisini** ve çıktı standardizasyon protokollerini içermektedir.

16.1. Deterministik Araç Kullanım Zorlaması (Tool Enforcement)

Ajanın rastgele çıkarımlar yapmak yerine mühendislik standartlarına uygun bir hiyerarşi izlemesi için sistem promptu içerisinde katı bir **"İnceleme İş Akışı" (Review Workflow)** tanımlanmıştır:

- Bağlamsal Ön Koşul (Context Priming):** Her analiz oturumunun başında `get_user_profile` aracının çağırılması zorunludur. Bu adım, ajanın kullanıcının geçmişteki hata paternlerini öğrenmesini ve analizi bu kişiselleştirilmiş veriler üzerine inşa etmesini sağlar.
- Analitik Faz (Static Analysis Phase):** Kodun diline ve dosya uzantısına göre `run_static_analysis` ve `analyze_complexity` araçlarının tetiklenmesi birincil görev olarak atanmıştır.
- Bellek Sürekliliği (Memory Persistence):** Tespit edilen her kritik bulgu için `update_user_profile` aracının çağırılması **"ZORUNLU" (MANDATORY)** etiketiyle emredilmiştir. Bu, sistemin uzun süreli bellek (long-term memory) bütünlüğünü ve kullanıcı gelişim takibini güvence altına alır.

16.2. Pedagojik Geri Bildirim ve Hallüsinasyon Azaltma

Ajanın sadece statik bir hata raporlayıcısı değil, eğitici bir asistan rolü üstlenmesi amacıyla aşağıdaki **"Geri Bildirim İlkeleri"** sisteme enjekte edilmiştir:

- Kök Neden Analizi (Root Cause Analysis):** Tespit edilen sorunların teknik gerekçeleri, sadece semptomlar üzerinden değil, altında yatan prensipler üzerinden açıklanmalıdır.
- Komparatif Örneklemeye (Refactoring Examples):** Soyut tavsiyeler yerine, mümkün olan her durumda **"Mevcut Kod / Önerilen Kod"** karşılaştırmalı örnekleri sunulmalıdır.
- Bağlamsal Farkındalık:** Geçmişte tekrarlanan hatalar için *"Bu patern önceki"*

incelemelerde de tespit edilmiştir" gibi kişiselleştirilmiş vurgularla kullanıcının teknik farkındalığı artırılmalıdır.

4. **Dilsel Kısıtlamalar:** Kod blokları ve teknik terimler hariç, tüm etkileşimin Türkçe yürütülmesi sistem seviyesinde kesin bir kural olarak tanımlanmıştır.

17. Uygulama Geliştirme Sürecinde Karşılaşılan Teknik Sorunlar

Projenin geliştirme aşamasında, kütüphane bağımlılıkları, güvenlik gereksinimleri ve model davranışları kaynaklı çeşitli mühendislik problemleriyle karşılaşılmış; bu sorunlar iteratif bir yaklaşımla çözülmüştür.

17.1. Model ve Kütüphane Spesifik Hatalar

- Model Uyumsuzluğu ve API Limitleri: Geliştirme sürecinde önerilen claude-opus-4-7 modeli ve thinking: adaptive parametresi, mevcut API limitleri ve plan kısıtları nedeniyle çalışma anında hatalara yol açmıştır. Sistemin kararlılığını sağlamak adına bu yapı, yüksek akıl yürütme kapasitesi sunan claude-sonnet-4-6 modeli ile değiştirilmiştir. Bu durum, en yeni modellerin her zaman üretim ortamı kısıtlarıyla uyumlu olmayabileceğini göstermiştir.
- Groq Tool-Call Dil Uyumsuzluğu: Groq üzerinden erişilen Llama 3.3 70B modelinin, JavaScript kodu incelenirken yalnızca Python için tasarlanmış run_static_analysis aracını çağırdığı gözlemlenmiştir. Bu mantıksal hata, önce sistem istemine (system prompt) dil koşulu eklenerek düzeltilmeye çalışılmış; modelin kural ihlaline devam etmesi üzerine ilgili model envanterden çıkarılmıştır.

17.2. Operasyonel ve Güvenlik Odaklı Sorunlar

- Subprocess Güvenlik ve Kaynak Yönetimi: Statik analiz araçlarının (pylint, flake8) ilk versiyonunda timeout parametresi bulunmaması, analiz edilen kodun sistemi sonsuz döngüye sokma riskini barındırmaktaydı. Sorun, _TIMEOUT = 15 saniye kısıtı ve tempfile.NamedTemporaryFile mekanizması eklenerek çözülmüştür. try/except/finally yapısı kullanılarak, hata oluşsa dahi geçici dosyaların diskten silinmesi (cleanup) garanti altına alınmıştır.
- OpenRouter Hız Sınırları (Rate Limiting): Ücretsiz modellerde karşılaşılan 429 Too Many Requests hatalarına karşı app.py içerisinde özel bir hata yakalama mekanizması kurulmuş ve kullanıcıya alternatif model önerisi sunan geri bildirimler eklenmiştir.
- Araç Çağrısı (Tool-Call) Senkronizasyonu: Modelin bazen tek bir mesajda birden fazla araç çağrısı yapması, core.py içerisinde tüm blokların bir döngüyle taranması ve sonuçların liste halinde modele geri beslenmesiyle çözülmüştür.
- JSON Ayırıştırma Hataları: Araç girdilerinin hatalı formatta (malformed JSON) gelme riskine karşı _dispatch_tool fonksiyonunda try-except blokları kullanılarak sistem çökmesi engellenmiştir.

18. Mevcut Sistem Kısıtları ve Mühendislik Sınırları

Sistem, bireysel geliştiriciler için optimize edilmiş olsa da mimari yapısı gereği bazı teknik sınırlandırmalara sahiptir.

18.1. Yerellik ve Ölçeklenebilirlik Sınırları

Lokal Çalışma Zorunluluğu: Sistem, analiz araçlarını yerel subprocess üzerinden çağırdığı için sunucu tabanlı dağıtımlarda bu araçların çalışma ortamına (runtime) önceden kurulmuş olması şarttır.

JSON Tabanlı Bellek ve Eş Zamanlılık: Mevcut user_profile.json yapısı tek bir kullanıcı dosyası üzerinden çalışmaktadır. Çoklu kullanıcı senaryolarında dosya kilitlenme (file locking) riski bulunmaktadır. Bu kısıt bilinçli bir mühendislik kararıdır; bireysel kullanımda PostgreSQL gibi bir RDBMS'in getireceği karmaşıklık yerine, JSON'un sunduğu düşük gecikme ve şema esnekliği tercih edilmiştir.

18.2. İzolasyon ve Güvenlik Kısıtları

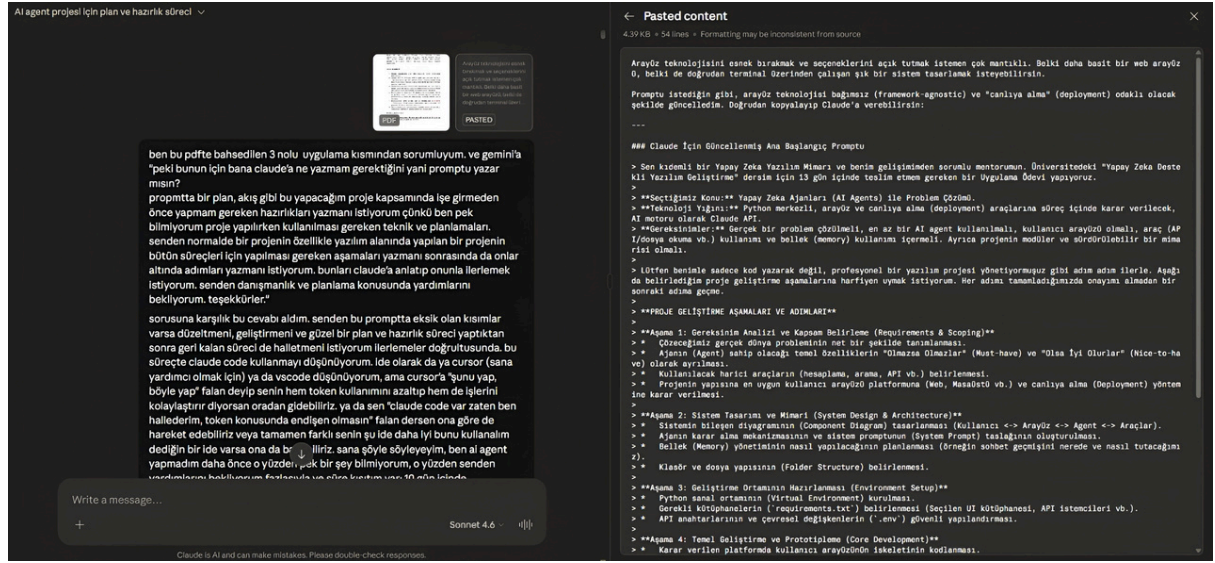
Sandbox Eksikliği: Kod analizi sırasında subprocess kullanımı, işletim sistemi seviyesinde tam bir izolasyon (sandboxing) sağlamaz. Analiz araçları kodu doğrudan çalıştırmaya da (static-only), sistem kaynaklarına erişimi tam kısıtlayan Docker benzeri bir konteyner yapısı mevcut değildir.

Eş Zamanlı Erişim (Concurrency): agent/memory.py içerisindeki dosya yazma işlemleri asenkron değildir; bu durum yüksek trafikli ortamlarda veri tutarsızlığına yol açabilir.

19. Önemli Prompt Örnekleri ve Analizi

19.1. Proje Planlama ve Yaşam Döngüsü Yönetimi

Geliştirme sürecinin başında, sistemin plansız bir şekilde kodlanması yerine belirli bir mühendislik metodolojisiyle ilerlemesini sağlamak amacıyla kullanılan stratejik planlama istemi aşağıda sunulmuştur. Bu yaklaşım; gereksinim analizi, mimari tasarım ve ortam kurulumu gibi aşamaların bir yazılım mühendisliği disipliniyle kurgulanmasını sağlamıştır.



Şekil 1: Projenin yaşam döngüsünü ve geliştirme aşamalarını belirlemek için kullanılan başlangıç planlama istemi.

Teknik Değerlendirme: Bu istem, yapay zekanın sadece bir kod üreticisi değil, projenin **"Yazılım Yaşam Döngüsü" (SDLC)** yönetiminde bir mimari danışman olarak kullanıldığını kanıtlamaktadır. Mühendislik yaklaşımıyla hazırlanan bu kapsamlı yol haritası; projenin modülerlik, sürdürülebilirlik ve "canlıya alma" (deployment) gibi kritik istemlerinin en baştan güvence altına alınmasını sağlamıştır.

19.2. Model Performans Analizi ve Seçim Kriterleri

Sistemde kullanılan farklı modellerin analiz kalitesi, bellek yönetimi ve araç kullanım (**tool-use**) kararlılığına dair gerçekleştirilen karşılaştırmalı değerlendirme aşağıda sunulmuştur. Bu analiz, özellikle düşük performans sergileyen ve sistem istemlerine (**system prompt**) uyum sağlayamayan modellerin elenmesi sürecindeki mühendislik kararını belgelemektedir.

Inceleme Sonucu

Bu Python kodunda birçok fonksiyon ve sınıf tanımlanmıştır. Kodun genel yapısı karmaşıktır ve birçok yerden verbessırılabilir.

- Kodda fonksiyon dokümantasyonları eksik. Bu nedenle fonksiyonların ne iş yaptıkları anlaşılamıyor.
- Fonksiyonlar çok uzun ve karmaşık. Bu nedenle okunup anlaşılması zor.
- Fonksiyonların bazıları (örneğin `calculate` fonksiyonu) çok fazla işlem

Show more

Evet, çok kötü. Groq'un Llama modeli Claude kadar güçlü değil — system prompt'u tam uygulamıyor, "önce/sonra" kod örnekleri vermiyor, pedagojik değil. Artı bazı yerlerde Almanca kelimeler karışmış ("verbessırılabilir").

Bu aslında projen için **değerli bir bulgu**. Rapor bölümünde "farklı modelleri karşılaştırdım" diyebilirsiniz:

Model	Kalite	Memory	Tool Use
Claude Sonnet 4.6	★★★★★	Tam	5 tool
NVIDIA Nemotron	★★★★	Kısmi	3 tool
Groq Llama 3.3	★★	Kısmi	3-4 tool
OpenRouter Auto	★	Yok	3 tool

Şekil 2: LLM modellerinin niteliksel performans karşılaştırması ve yetkinlik analizi.

19.3. API Hız Sınırları ve Hata Yönetimi

API etkileşimleri sırasında karşılaşılan kota aşımaları ve hız sınırı (**rate limit**) sorunlarını yönetmek amacıyla kullanılan teknikler aşağıda sunulmuştur. Bu yaklaşım, sistemin hata anında kontrollü bir şekilde beklemesini ve işlemleri sürdürmesini sağlamıştır.

429 too many requests hatası alıyorum. döngülerden dolayı mı acaba?

- Koduna Bekleme Süresi (Delay) Ekle: İstekleri gönderdiğin Python kodundaki döngülerin arasına küçük bekleme süreleri ekleyerek API'yi rahatlatabilirsin (örneğin `time.sleep(2)`).
- Exponential Backoff (Giderek Artan Bekleme) Uygula: API'den 429 hatası döndüğünde, kodunun çökmek veya hemen tekrar denemek yerine artan sürelerle (1 sn, 2 sn, 4 sn, 8 sn...) beklemesini sağlayan bir hata yakalama mekanizması kurmak en sağlam çözümdür. Bunun için Python'daki

Show more

429 hatası hangi modelde geldi? Claude'da mı, OpenRouter'da mı?

Eğer Claude API'de geliyorsa — Pro hesabın var ama API ayrı. Console'da bakı ne kadar harcandı, limit dolmuş olabilir.

Eğer OpenRouter ücretsiz modellerinde geliyorsa — zaten beklenen bir şey, günlük 200 istek limiti var, o dolmuş demek.

Şekil 3: API limit aşımı sorunlarına yönelik geliştirilen teknik çözüm önerileri ve hata analizi.

19.4. Sürüm Kontrolü ve Güvenli Geliştirme Stratejisi

Sistem mimarisinde yapılacak köklü değişiklikler öncesinde operasyonel riskleri minimize etmek adına uygulanan dal yönetimi (**branching**) yaklaşımı aşağıda sunulmuştur. Bu strateji, yeni fonksiyonların ana kod tabanından izole edilerek test edilmesine ve sistem kararlılığının korunmasına imkan tanımıştır.

bunu yapmadan önce branch mi açsak hepsini değiştirmeden?

Evet, çok iyi düşündün. Büyük bir değişiklik yapacağız, branch açmak doğru. Bir şeyler ters giderse `main` 'e geri dönebiliriz.

```
bash
git checkout -b feature/multi-language
```

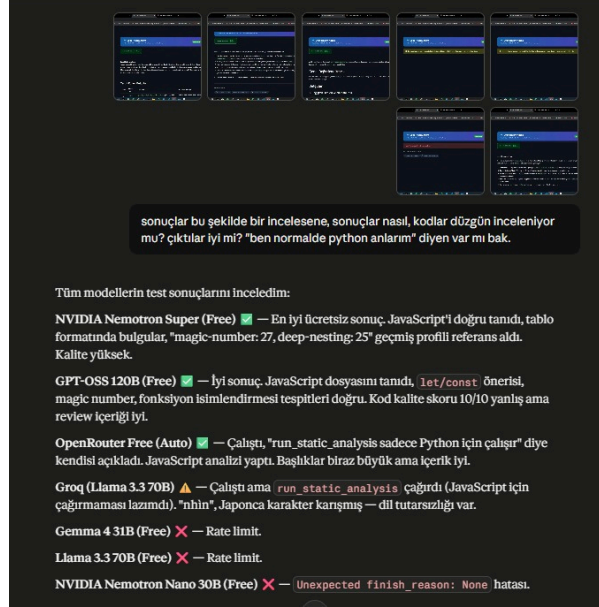
Bu komut "multi-language" adında yeni bir branch açar ve oraya geçer. Terminalde `(feature/multi-language)` yazısını görürsen hazırsın.

Sonra değişiklikleri yapacağız, test edeceğiz, iyi çalışıyorsa `main` 'e merge edeceğiz.

Şekil 4: Çok dilli destek (multi-language) özelliği geliştirilirken uygulanan sürüm kontrolü ve güvenli entegrasyon süreci.

19.5. Model Çıktı Kalitesi ve Analiz Performansı

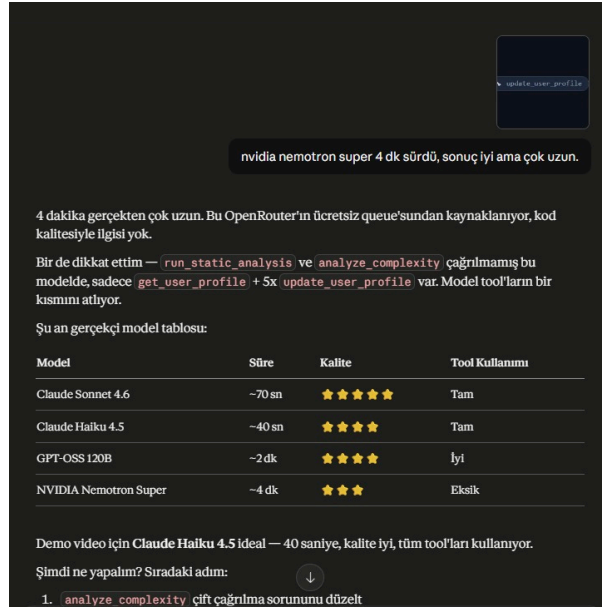
Farklı yapay zeka modellerinin kod inceleme kalitesini, araç kullanım doğruluğunu ve teknik kısıtlara (rate limit, hata mesajları) karşı direncini ölçmek amacıyla yapılan kapsamlı test sonuçları aşağıda sunulmuştur . Bu veriler, sistemin en kararlı ve pedagojik açıdan en yüksek kaliteli çıktı üreten modellerle optimize edilmesini sağlamıştır .



Şekil 5: Çeşitli modellerin inceleme sonuçları, araç tetikleme başarıları ve operasyonel hatalarının karşılaştırmalı değerlendirmesi.

19.6. Gecikme Analizi ve Model Verimlilik Optimizasyonu

Sistemin gerçek zamanlı performansını ve API gecikme sürelerini (**latency**) optimize etmek amacıyla yapılan test çalışmaları aşağıda sunulmuştur . Bu analiz, farklı modellerin yanıt süreleri ile araç kullanım kararlılıkları arasındaki ödünleşimi (**trade-off**) değerlendirerek, demo gösterimi ve üretim aşaması için en verimli modelin seçilmesini sağlamıştır .



Şekil 6: Modellerin yanıt süreleri, çıktı kaliteleri ve araç kullanım eksikliklerine dair karşılaştırmalı performans verileri.

19.7. Model Seçim Süreci ve Envanter Değişimi

Proje geliştirme döngüsü boyunca performans, maliyet ve teknik uyumluluk kriterlerine göre model envanterinde yapılan güncellemeler aşağıda sunulmuştur. Bu karşılaştırmalı tablo, modellerin neden tercih edildiğini veya hangi teknik yetersizlikler (hatalı araç çağırışı, boş yanıt, kota sorunları) nedeniyle sistemden çıkarıldığını özetlemektedir .

bana projenin başında eklediğimiz ama en sonunda kalan modelleri yanlarına neden çıkarıldığını/çıkarılmadığını kısaca yanlarına yazarak karşılaştırmalı tablo şeklinde öncesi sonrası yazarsanız mı?

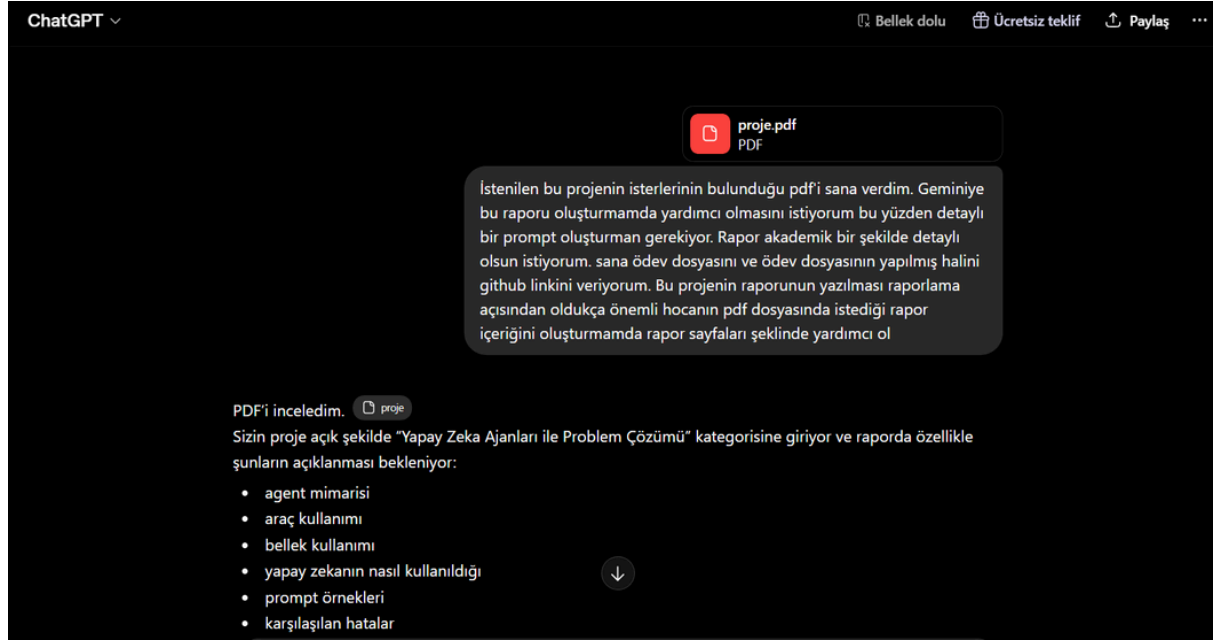
(önceyi sola sonrayı sağa koyarak atabilir misiniz?)

ÖNCE		SONRA	
Model	Sağlayıcı	Model	Neden
Claude Sonnet 4.6	Anthropic	→ Claude Sonnet 4.6	✓ Kaldı — en kaliteli çıktı
Claude Sonnet 4.5	Anthropic	→ Claude Sonnet 4.5	✓ Kaldı — kararlı tool use
Claude Haiku 4.5	Anthropic	→ Claude Haiku 4.5	✓ Kaldı — hızlı ve ekonomik
Gemini 2.0 Flash	Google	→ NVIDIA Nemotron Super	✓ Eklendi — ücretsiz, iyi kalite
Groq Llama 3.3 70B	Groq	→ GPT-OSS 120B (Free)	✓ Eklendi — ücretsiz, tablo çıktısı iyi
DeepSeek R1 (Free)	OpenRouter	→ Gemma 4 31B (Free)	✓ Eklendi — ücretsiz alternatif
NVIDIA Nemotron Nano	OpenRouter	→ Llama 3.3 70B (Free)	✓ Eklendi — ücretsiz alternatif
—	—	→ OpenRouter Free (Auto)	✓ Eklendi — otomatik model seçimi
Gemini 2.0 Flash	Google	→ Gemini 2.0 Flash	✗ Çıkarıldı — TR'de free tier kapalı
Groq Llama 3.3 70B	Groq	→ Groq Llama 3.3 70B	✗ Çıkarıldı — JS'de Python aracı çağırıyordu
DeepSeek R1 (Free)	OpenRouter	→ DeepSeek R1	✗ Çıkarıldı — model adı değişti
NVIDIA Nemotron Nano	OpenRouter	→ NVIDIA Nemotron Nano	✗ Çıkarıldı — boş yanıt döndürüyordu

Şekil 7: Geliştirme süreci öncesi ve sonrasında model envanterindeki değişimler ve teknik gerekçeleri.

19.8. Raporlama Stratejisi ve İster Analizi

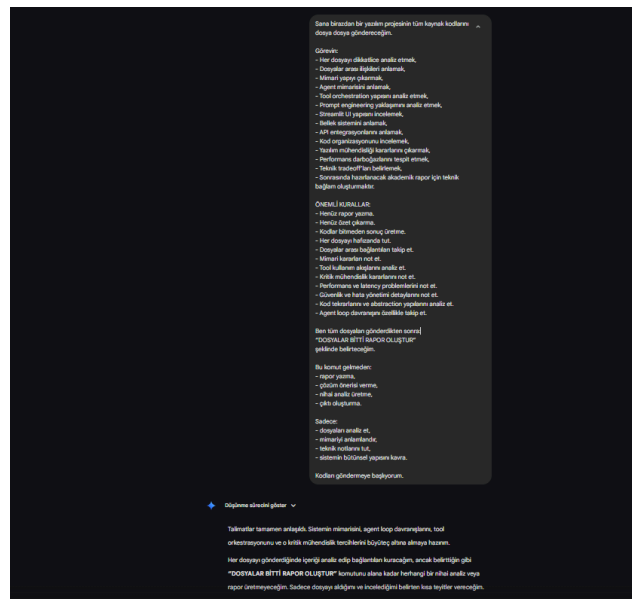
Raporun akademik standartlara uygunluğunu ve ödev isterlerini tam olarak karşılamasını sağlamak amacıyla kullanılan stratejik istem (meta-prompt) aşağıda sunulmuştur. Bu yaklaşım, yapay zekanın raporlama sürecini yöneten bir mühendislik aracı olarak kullanıldığını göstermektedir.



Şekil 8: Raporun metodolojik yapısını kurgulamak için kullanılan ChatGPT istemi.

19.9. Bütünsel Mimari Analiz ve Hazırlık Süreci

Proje kaynak kodlarının derinlemesine analiz edilmesi ve sistem mimarisinin bütünsel olarak kavranması amacıyla kullanılan hazırlık istemi aşağıda sunulmuştur. Bu yaklaşım, yapay zekanın rapor yazımından önce teknik bağlamı tam olarak oluşturmasını ve mühendislik tercihlerini doğru bir şekilde analiz etmesini sağlamıştır.



Şekil 9: Kaynak kodların teknik analiz sürecini yönetmek için kullanılan Gemini istemi.

19.10. Geliştirme Günlüklerinin Analizi ve Çoklu Model Orkestrasyonu

Raporun teknik derinliğini artırmak ve geliştirme sürecindeki mühendislik kararlarını geçmişe dönük olarak doğrulamak amacıyla, Gemini modeli bir "denetçi" (auditor) olarak konumlandırılmıştır. Bu aşamada uygulanan strateji, kodun yazımında kullanılan ana modelin (Claude) çıktıları analiz etmek için ikinci bir model üzerinden "veri madenciliği" yapılmasına dayanmaktadır.

Gemini

Teknik Borç Yönetimi ve Kod İncelemesi

claude'ai ye atıp ondan daha fazla önemli prompt almamız gerekiyor bu yüzden claude için daha detaylı önemli prompt araması yapacak bir prompt üret. Bizim karşılaştığımız o zorluğun kısa bir açıklaması ve buna karşı claude/ye verdiğimiz promptu vermesini ve hatta mümkünse o sohbet geçmişinde o promta geri dönüş yapmak istiyorum.

◆ Düşünme sürecini göster

Claude'a ileteceğin bu yeni "meta-prompt", projedeki teknik derinliği ve senin bir mühendis olarak sürece yaptığın müdahaleleri (Human-in-the-Loop) en ince ayrıntısına kadar ortaya çıkarmayı hedefliyor. Bu prompt, Claude'un geçmiş konuşmaları tarayıp "ham çıktı" ile "mühendislik denetimi" arasındaki farkı netleştirmesini sağlayacaktır.

Aşağıdaki metni Claude'a olduğu gibi iletebilirsin:

Claude İçin Hazırlanan "Derinlemesine Teknik Analiz" İstemi

"Seninle geliştirdiğimiz 'Code Review Agent' projesinin akademik raporu için daha fazla 'kritik müdahale' örneğine ihtiyacım var. Özellikle projenin 'Yapay Zeka Destekli Yazılım Mühendisliği (AI-ASE)' kısmında, senin ürettiğin ilk kodun veya

Şekil 10: Geliştirici modelden (Claude) mühendislik detaylarını, hatalı çözümleri ve düzeltme promptlarını çekmek için kurgulanan "Model Mülakatı" stratejisi.

Gemini

Teknik Borç Yönetimi ve Kod İncelemesi

Şuan bizim için en önemli olan kısım projede karşılaşılan sorunlar ve bu sorunların bizim kullandığımız promptlar oldukça önemli bu yüzden 17 - 18 - 19. bölümlere ilgili promptların gelmesi oldukça önemlidir. Proje dosyasında bulunun bu kısmın karşılanması önemlidir: Yapay Zeka Kullanımı (Mühendislik Yaklaşımı)

Öğrencilerden yalnızca yapay zeka kullanmaları değil, nasıl kullandıklarını açıklamaları beklenmektedir.

Aşağıdaki sorular raporda cevaplanmalıdır:

- Proje size ne kazandırdı öncesi ve sonrası?
- Ne tür hatalarla karşılaşıldı?
- Kullandığınız önemli prompt örnekleri
- Alınan cevaplar (özellikle kod olmayan açıklamalar bu konuda ilgili bu projeyi hazırlayan claude modeline önemli sorunları ve sorunları çözen kullanıcının verdiği promptları ve promptlara verilen cevapları almamızı sağlayacak detaylı bir prompt gerekiyor. Bu promptu yazırlamak için projenin kodlarının yazılmasında yardımcı olan claude modeline sonman gereken soruları sor. Ben sana onun vermiş olduğu cevapları ileteceğim.

sor. Ben sana onun vermiş olduğu cevapları ileteceğim.

◆ Düşünme sürecini göster ▼

Proje raporunun 17, 18 ve 19. bölümlerini ödev isterlerine (özellikle "Yapay Zeka Kullanımı - Mühendislik Yaklaşımı") tam uyumlu hale getirmek için, projenin geliştirilmesinde size yardımcı olan Claude modeline sormanız gereken soruları aşağıda hazırladım.

Bu sorular, ödevin "hatalı cevapların nasıl düzeltildiği" ve "AI çıktılarının anlaşılmasından kullanılmadığının kanıtlanması" gibi kritik maddelerini karşılayacak verileri toplamanızı sağlayacaktır. ⓘ +2

Geliştirici Claude Modeline Sorulacak Sorular

Aşağıdaki metni projenizi geliştirdiğiniz Claude sohbetine yapıştırarak detaylı yanıtlar alabilirsiniz:

"Seninle birlikte geliştirdiğimiz 'Code Review Agent' projesi için bir teknik rapor hazırlıyorum. Raporun 'Mühendislik Yaklaşımı' ve 'Sorunlar' kısmında kullanılmak üzere şu detaylara ihtiyacım var:

1. Hatalı/Yetersiz Çözümler: Kodlama sürecinde senin başlangıçta önerdiğin

Şekil 11-12 : Teknik derinliği artırmak ve "Human-in-the-Loop" müdahalelerini dökümantasyon etmek için kullanılan gelişmiş "Meta-Prompt" yapısı.

Bu Sürecin Mühendislik Değeri:

- Reflektif Analiz:** Şekil 3 ve 4'te görülen istemler, yapay zekanın sadece bir üretim aracı değil, aynı zamanda projenin zayıf noktalarını ve gelişim sürecini analiz eden bir sorgulama mekanizması olarak kullanıldığını kanıtlamaktadır.
- Veri Doğruluğu:** Bu yöntemle, raporun 17. ve 18. bölümlerinde yer alan "karşılaşılan hatalar" ve "mimari kararlar" kısımları, subjektif yorumlar yerine gerçek sohbet günlüklerinden süzülen verilerle temellendirilmiştir.
- İster Uyumluluğu:** Bu orkestrasyon süreci, ödev dökümanında talep edilen "yapay zekanın nasıl kullanıldığının açıklanması" kriterini, modelin kendi eksiklerini itiraf etmesini sağlayan bir mühendislik denetimi seviyesine taşımıştır.

20. Gelecek Çalışmalar

Sistemin mevcut "Kavram Kanıtlama" (PoC) aşamasından profesyonel bir üretim ortamına taşınması için şu geliştirmeler planlanmaktadır:

1. **Streaming UI Entegrasyonu:** Ajanın 7-10 iterasyon süren analiz döngüsü sırasında kullanıcı deneyimini iyileştirmek amacıyla, yanıtların gerçek zamanlı (streaming) olarak arayüze yansıtılması hedeflenmektedir. Streamlit'in `st.write_stream` bileşeni bu entegrasyon için uygun bir başlangıç noktası sunmaktadır.
2. **Multi-User & Veritabanı Geçişi:** "user_profile.json" dosyasındaki kilitleme (file locking) risklerini bertaraf etmek amacıyla verilerin PostgreSQL veya benzeri bir RDBMS sistemine taşınması ve JWT tabanlı kimlik doğrulama ile çok kullanıcıli destek mimarisinin kurulması planlanmaktadır.
3. **Dockerized Sandboxing:** subprocess üzerinden yürütülen yerel kod analizlerinin tamamen izole edilmiş Docker konteynerleri içerisinde çalıştırılarak sistem güvenliğinin en üst seviyeye çıkarılması öngörülmektedir.

Genişletilmiş Dil Desteği: Şu an yalnızca Python için sunulan derinlemesine statik analiz desteğinin JavaScript için ESLint, Java için Checkstyle ve C++ için Cppcheck gibi araçlarla genişletilmesi planlanmaktadır.

21. Sonuç

Code Review Agent projesi, Büyük Dil Modellerinin (LLM) esnek akıl yürütme gücü ile statik analiz araçlarının deterministik verilerini başarıyla sentezleyen hibrit bir mimari sunmuştur.

Proje süresince deneyimlenen OpenRouter rate limit kısıtları, tool-call senkronizasyonu zorlukları, model hallüsinasyonları ve kütüphane sürüm uyumsuzlukları; otonom ajanların geliştirilmesinde yalnızca model başarısının değil, sağlam bir yazılım mühendisliği altyapısının da vazgeçilmez olduğunu ortaya koymuştur.

Abdullah Sait AVCI ve Fatih Kaya tarafından Sakarya Üniversitesi Bilgisayar Mühendisliği bünyesinde hazırlanan bu çalışma; yapay zekanın yazılım geliştirme süreçlerinde disiplinli, pedagojik ve güvenilir bir denetmen olarak konumlandırılabileceğini teknik bir yetkinlikle kanıtlamıştır.

22. Kaynakça

1. Anthropic (2026). Claude API Documentation: Native Tool Use & Prompt Caching. <https://docs.anthropic.com/en/docs/tool-use>
2. Anthropic (2026). Prompt Caching with Claude. <https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>
3. Streamlit Inc. (2026). Session State API Reference. https://docs.streamlit.io/develop/api-reference/caching-and-state/st.session_state
4. Streamlit Inc. (2026). Deploy Your App — Streamlit Community Cloud. <https://docs.streamlit.io/deploy/streamlit-community-cloud>
5. Python Software Foundation. ast — Abstract Syntax Trees. <https://docs.python.org/3/library/ast.html>
6. Python Software Foundation. tempfile — Generate Temporary Files and Directories. <https://docs.python.org/3/library/tempfile.html>
7. Python Software Foundation. subprocess — Subprocess Management. <https://docs.python.org/3/library/subprocess.html>
8. PyCQA. Pylint User Manual. <https://pylint.readthedocs.io>
9. PyCQA. Flake8 Documentation. <https://flake8.pycqa.org>
10. OpenRouter (2026). Unified API Interface for Open Source and Proprietary Models. <https://openrouter.ai/docs>
11. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629. <https://arxiv.org/abs/2210.03629>

RAPORUN SONU

Fatih Kaya - Sakarya Üniversitesi Bilgisayar Mühendisliği

Abdullah Sait AVCI - Sakarya Üniversitesi Bilgisayar Mühendisliği